[NAME OF UNIVERSITY]
[College / Department]
Dumaguete City, Negros Oriental, Philippines

GUIDEAI: AN OFFLINE-FIRST MOBILE ASSISTIVE APPLICATION FOR VISUALLY IMPAIRED PEDESTRIANS USING ON-DEVICE YOLO11 OBSTACLE DETECTION, VOICE GUIDANCE, AND EMERGENCY SMS

A Capstone Project
Presented to the Faculty of the [College / Department]
[Name of University]

In Partial Fulfillment of the Requirements for the Degree
[Bachelor of Science in Information Technology / Computer Science]

by

[Proponent 1: Full Name]
[Proponent 2: Full Name]
[Proponent 3: Full Name]
[Month Year]

APPROVAL SHEET

This capstone project entitled "GuideAI: An Offline-First Mobile Assistive Application for Visually Impaired Pedestrians Using On-Device YOLO11 Obstacle Detection, Voice Guidance, and Emergency SMS," prepared and submitted by [Proponent names], in partial fulfillment of the requirements for the degree [Degree Program], has been examined and is recommended for acceptance and approval for ORAL EXAMINATION.

[Name of Adviser]
Capstone Adviser

PANEL OF EXAMINERS

Approved by the Committee on Oral Examination with a grade of _____.

[Name of Panelist]
Chairperson

[Name of Panelist]
Member

[Name of Panelist]
Member

Accepted and approved in partial fulfillment of the requirements for the degree [Degree Program].

[Name of Dean / Program Chair]
[Title]
Date: _____

ABSTRACT

Visually impaired pedestrians in Philippine cities face hazards that a white cane alone cannot always reveal in time: open drainage canals, potholes, stairs, utility poles in the middle of sidewalks, and tricycles, jeepneys, and motorcycles that share narrow roads with people on foot. Most available electronic travel aids are expensive, depend on a constant internet connection, or were designed for other countries. This capstone project developed GuideAI, an Android application that turns an ordinary smartphone into an assistive device for blind and visually impaired pedestrians in Dumaguete City. The application detects obstacles in real time with a YOLO11 nano object detection model that runs entirely on the phone, announces them through short spoken warnings and vibration, reads printed text with on-device optical character recognition, gives walking directions using OpenStreetMap routing, and sends emergency text messages with the user's location directly from the phone's SIM card. An optional online layer uses Google Gemini to answer free-form voice questions and to name objects outside the detector's classes. The detector was trained on 25,327 images (22,405 training, 1,856 validation, 1,066 test) from nine public datasets of Philippine vehicles, potholes, open drainage, stairs, and sidewalk obstacles, plus 5,000 everyday scenes from the COCO dataset, covering 25 classes. On the held-out test set, the model currently deployed in the application reached a precision of 0.697, a recall of 0.349, and an mAP50 of 0.386; a retrained model reached an mAP50 of 0.480 with a recall of 0.477, performing best on tricycles, buses, jeepneys, people, and cars (mAP50 0.83 to 0.93) and weakest on furniture, animals, and street fixtures that had little training data. On a mid-range phone (Honor X9a 5G, Android 13) the application analyzed 2.5 to 3.0 camera frames per second with 135 to 215 milliseconds of model time per frame. All 236 automated software tests passed, and field tests on the phone uncovered and led to the correction of a false "open drain" warning when the camera was covered. User acceptance testing with visually impaired participants using the ISO/IEC 25010 quality model is [to be conducted / results summarized here]. The study shows that real-time, offline obstacle warnings are feasible on an ordinary smartphone, and it identifies more local training images and the planned smart cane as the main paths to better accuracy.

Keywords: assistive technology, visual impairment, YOLO11, on-device object detection, obstacle detection, text-to-speech, mobile application, Philippines

ACKNOWLEDGEMENT

The proponents wish to express their sincere gratitude to the following people who helped make this capstone project possible:

- [Name of Adviser], our capstone adviser, for the guidance, patience, and expert advice throughout the project;
- the panel of examiners, [names], for their valuable comments and suggestions;
- [Name of partner organization / school for the blind], and the visually impaired participants who shared their experiences and tested the application;
- our families and friends, for their constant support and encouragement; and
- above all, the Almighty God, for the wisdom and strength to finish this work.

[Proponent names]

TABLE OF CONTENTS

APPROVAL SHEET i

ABSTRACT ii

ACKNOWLEDGEMENT iii

TABLE OF CONTENTS iv

LIST OF TABLES vii

LIST OF FIGURES viii

CHAPTER I THE PROBLEM AND ITS BACKGROUND 1

Introduction 1

Background of the Study 1

Statement of the Problem 2

Objectives of the Study 3

Significance of the Study 4

Scope and Delimitation 4

Definition of Terms 5

CHAPTER II REVIEW OF RELATED LITERATURE AND STUDIES 7

Visual Impairment and Mobility 7

Assistive Navigation Technologies 7

Deep Learning Object Detection 8

Multi-Object Tracking 8

Vision-Language Models 9

Text Recognition, Routing, and Speech 9

Accessibility Standards and Philippine Laws 9

Related Local Studies 10

Synthesis 10

Conceptual Framework 10
CHAPTER III METHODOLOGY 12
Research Design 12
Software Development Methodology 12
System Architecture 13
System Diagrams 13
Hardware and Software Requirements 14
Dataset Collection and Preparation 14
Model Training 15
Model Export and Deployment 16
Obstacle Detection and Warning Pipeline 16
Voice Warning Design 18
Supporting Modules 19
Accessibility Design 20
Testing and Evaluation 21
Ethical Considerations 22
CHAPTER IV RESULTS AND DISCUSSION 23
The Developed System 23
Model Performance 25
On-Device Performance 28
Software Testing 29
Field Testing on the Phone 29
User Acceptance Evaluation 32
Discussion 32
CHAPTER V SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS 33

Summary of Findings 33
Conclusions 33
Recommendations 34
REFERENCES 35
APPENDIX A: EVALUATION QUESTIONNAIRE (ISO/IEC 25010) 37
APPENDIX B: VOICE COMMANDS 38
APPENDIX C: SMART CANE BLUETOOTH PROTOCOL (SUMMARY) 39
APPENDIX D: USER MANUAL 40
APPENDIX E: DETECTION CLASSES AND SPOKEN WORDS 41
APPENDIX F: DATASET SOURCES 42
APPENDIX G: SYSTEM USE CASES AND DATA MODEL 43
APPENDIX H: CURRICULUM VITAE 45

LIST OF TABLES

Table 1. Input-Process-Output conceptual framework of the study 10
Table 2. Hardware used in the study 14
Table 3. Software and technologies used 14
Table 4. Image sources of the merged dataset 15
Table 5. Training settings 15
Table 6. Voice warning rules 18
Table 7. Likert scale and interpretation 22
Table 8. Overall detection accuracy on the test set (1,066 images) 26
Table 9. Per-class test results, sorted by v2 mAP50 26
Table 10. On-device performance (Honor X9a 5G, Android 13) 29
Table 11. Areas covered by the automated tests 29
Table 12. Field test cases and results 29
Table 13. Summary of the ISO/IEC 25010 evaluation (to be filled in) 32
Table 14. Voice command examples 38
Table 15. Cane service 6e7a0001-4f1b-4c1d-9b3a-2a6e8c9d0b00 39
Table 16. Cane hazard thresholds (shared by the firmware and the application) 39
Table 17. The 25 detection classes 41
Table 18. Description of the main use cases 43

LIST OF FIGURES

Figure 1. System architecture of GuideAI 13
Figure 2. Validation mAP50 per epoch for the two training runs 16
Figure 3. Obstacle detection and warning pipeline 17
Figure 4. Home, voice command, and settings screens 23
Figure 5. Obstacle detection screen in three situations (field test, 17 October 2026) 24
Figure 6. Navigation, text reader, and emergency SOS screens 25
Figure 7. Per-class test mAP50 of the two models 27
Figure 8. Normalized confusion matrix of model v1 on the validation set 28
Figure 9. Objects found and named by the online AI layer on sample test photos 31
Figure 10. Use-case diagram of GuideAI 43
Figure 11. Entity-relationship diagram of GuideAI 44

CHAPTER I
THE PROBLEM AND ITS BACKGROUND

Introduction

Independent mobility is one of the greatest challenges faced by people who are blind or visually impaired. Walking to school, to the market, or to a jeepney stop requires knowing where obstacles are, where the path drops or rises, and where vehicles are coming from. The World Health Organization (2019) estimates that at least 2.2 billion people worldwide have a near or distance vision impairment, and Bourne et al. (2017) estimated 36 million people to be blind in 2015, a number expected to rise as populations age. For these people, the white cane and the guide dog remain the main mobility aids. The white cane is reliable for objects on the ground within its reach, but it cannot detect obstacles at chest or head height, it gives no warning about moving vehicles, and it cannot read signs.

At the same time, the smartphone has become an everyday device even in low- and middle-income countries. Modern phones contain a camera, a processor able to run neural networks, a microphone, a speaker, a vibration motor, GPS, and a cellular modem. Recent progress in lightweight object detection models, especially the YOLO ("You Only Look Once") family (Redmon et al., 2016; Jocher & Qiu, 2024), makes it possible to recognize objects in a camera image within a fraction of a second on the phone itself, without sending the image to a server.

This capstone project, GuideAI (internally called BUTA APP, from the Cebuano and Tagalog word for "blind"), combines these capabilities into a single application designed from the start for blind users in the Philippines. It warns the user about obstacles and vehicles in front of them, reads text aloud, gives walking directions, and sends emergency text messages, and it keeps its safety functions working without an internet connection.

Background of the Study

Sidewalks in Philippine cities such as Dumaguete are often narrow, uneven, or missing. Pedestrians must share space with parked motorcycles, vendor carts, utility

poles, construction materials, and open drainage canals. Tricycles and jeepneys, which are common public utility vehicles in the country, frequently stop or turn close to people on foot. These conditions differ from those assumed by many assistive navigation products developed abroad, whose object detectors are usually trained on images of European or American streets and do not know local vehicles such as the tricycle or the jeepney.

Existing assistive applications also tend to depend on a steady internet connection, either because the image recognition runs on a cloud server or because the user must log in to an account. Mobile data in the Philippines can be intermittent, and many users rely on prepaid load that can run out. A safety warning that arrives several seconds late, or not at all because the connection dropped, is of little use to a person about to step into a canal. Emergency features that send alerts over the internet fail in exactly the situations where help is needed most.

These observations led the proponents to develop a mobile application whose core safety features run on the phone itself, which is trained on images of local hazards and vehicles, which can be operated without looking at the screen, and which uses the internet only for optional extras. The project also defines the communication protocol for a companion smart cane based on the ESP32 microcontroller, to be added when the hardware is built.

Statement of the Problem

This study aimed to design, develop, and evaluate a mobile assistive application that helps visually impaired pedestrians detect and avoid obstacles and move around more safely and independently. Specifically, it sought to answer the following questions:

1. What hazards and obstacles commonly encountered by visually impaired pedestrians in Dumaguete City can be detected by a smartphone camera and an on-device object detection model?
2. How can an object detection model be trained and optimized to recognize local hazards and vehicles, such as open drainage, potholes, stairs, tricycles, and jeepneys, and run in real time on a mid-range Android phone without an internet connection?

3. How can detected obstacles be communicated to a blind user through speech and vibration in a way that is timely, prioritized, and not overwhelming?
4. What supporting functions (text reading, walking navigation, voice commands, and emergency messaging) should the application provide, and how can they be made fully usable without sight?
5. How accurate and fast is the developed system, as measured by precision, recall, mean average precision (mAP), and on-device processing speed?
6. How do visually impaired users and IT experts evaluate the application in terms of the ISO/IEC 25010 quality characteristics of functional suitability, performance efficiency, usability, reliability, and security?

Objectives of the Study

General Objective

To develop GuideAI, an offline-first Android application that uses on-device YOLO11 object detection, voice guidance, and emergency SMS to help visually impaired pedestrians in Dumaguete City walk more safely and independently.

Specific Objectives

1. To gather and prepare a labeled image dataset of hazards, vehicles, and obstacles relevant to Philippine streets, including tricycles, jeepneys, potholes, open drainage, stairs, poles, barriers, and carts;
2. To train a lightweight YOLO11 nano model on this dataset and deploy it on an Android phone so that it runs without an internet connection;
3. To design a hazard warning system that estimates the direction and approximate nearness of each obstacle, confirms detections across camera frames, and speaks short prioritized warnings with vibration feedback;
4. To implement accessible supporting modules: on-device text reading, walking navigation with spoken turn-by-turn directions, bilingual voice commands, and an emergency SOS that sends SMS directly from the SIM card;
5. To evaluate the detection model using precision, recall, mAP50, and mAP50-95 on a held-out test set, and to measure the processing speed of the application on a real phone; and

6. To evaluate the application with visually impaired users and IT experts using a questionnaire based on ISO/IEC 25010.

Significance of the Study

The results of this study are expected to benefit the following:

- **Visually impaired pedestrians.** They gain an affordable tool, running on a phone they may already own, that warns them about obstacles beyond the reach of the cane, reads text for them, guides them to places, and lets them call for help even without mobile data.
- **Families and caregivers.** Emergency text messages with a map link let them locate and assist a family member quickly.
- **Schools and organizations for the blind.** The application can support orientation and mobility training and give learners more confidence when travelling alone.
- **Local government units.** The hazards most often detected (open drainage, potholes, obstructed sidewalks) point to infrastructure that needs attention, in support of accessibility laws such as Republic Act No. 7277 and its amendments.
- **Future researchers.** The dataset preparation scripts, training procedure, evaluation results, and the documented smart cane protocol can serve as a starting point for related studies.
- **The proponents.** The project applies knowledge in mobile development, machine learning, and human-centered design to a real social problem.

Scope and Delimitation

The study covers the design, development, and evaluation of the GuideAI Android application with the following features:

- real-time obstacle detection with the phone's back camera using a YOLO11 nano model with 25 classes, running on the phone (person, bicycle, car, motorcycle, tricycle, jeepney, bus, truck, dog, cat, cow, horse, pothole, open drainage, stairs, pole, barrier, bump, cart, bench, chair, fire hydrant, potted plant, traffic light, and stop sign);

- spoken warnings and vibration, with the direction of the obstacle (left, ahead, right) and a qualitative nearness band, prioritized by danger;
- on-device text reading using Google ML Kit;
- walking navigation using OpenStreetMap geocoding and OSRM routing, with spoken turn-by-turn directions;
- voice commands with English and Tagalog keywords offline, and an optional Google Gemini assistant that also understands Cebuano and free-form questions when online;
- optional naming of objects outside the 25 classes by Google Gemini when online;
- an emergency SOS with a five-second cancellable countdown that sends an SMS with a Google Maps link directly from the SIM card to saved contacts;
- a "Teach My AI" feature that saves pictures for later labeling and retraining on a computer; and
- a Bluetooth Low Energy protocol for a companion ESP32 smart cane, with a simulated cane data source in the application.

The study is delimited as follows. The application runs on Android phones only. Obstacle detection depends on lighting and camera position and works best in daylight with the phone held or worn at chest height facing forward. The model only recognizes the 25 classes listed; other objects are not detected by the offline model, although the online Gemini layer may name them. Distances are estimated from the size and position of objects in the image under an assumed camera height and tilt and are therefore approximate; the application speaks only nearness bands ("very close", "near", "moderate distance", "far") rather than exact meters. The smart cane hardware was not built within the study period; only its communication protocol and the application side were implemented and tested with simulated readings. The application is an aid to, not a replacement for, the white cane, guide dog, and orientation and mobility training. It is not a certified medical or navigation device.

Definition of Terms

The following terms are defined operationally as used in this study:

- **Bounding box.** The rectangle that an object detection model draws around an object it recognizes.

- **Confidence score.** A number from 0 to 1 that the model gives each detection, showing how sure it is.
- **Edge (on-device) inference.** Running a neural network on the phone itself instead of on a server.
- **EAS Update (over-the-air update).** A way of delivering new application code to installed phones without reinstalling the application.
- **Gemini.** A family of multimodal AI models by Google, used online in GuideAI for the voice assistant and for naming unfamiliar objects.
- **mAP50.** Mean average precision at an intersection-over-union threshold of 0.50; the main accuracy measure for object detectors. mAP50-95 averages this over thresholds from 0.50 to 0.95.
- **OCR (optical character recognition).** Converting printed text in an image into machine-readable text.
- **OSRM.** Open Source Routing Machine, a route planning engine that uses OpenStreetMap data.
- **Precision.** The share of the model's detections that are correct.
- **Recall.** The share of the real objects in the images that the model finds.
- **TFLite (TensorFlow Lite).** A compact model format for running neural networks on mobile devices.
- **Text-to-speech (TTS).** Converting text into spoken audio; used for all warnings and messages.
- **TalkBack.** The screen reader built into Android, used by blind people to operate a phone.
- **Visually impaired.** Having low vision or blindness that limits daily activities even with correction.
- **Walking path.** In this study, the central part of the camera image, which the user is walking into.
- **YOLO11n.** The nano (smallest) version of the YOLO11 object detection model by Ultralytics.

CHAPTER II
REVIEW OF RELATED LITERATURE AND STUDIES

This chapter presents literature and studies related to visual impairment, assistive navigation technologies, object detection, and the other technologies used in GuideAI, which guided the design of the system.

Visual Impairment and Mobility

The World Health Organization (2019), in its World Report on Vision, states that at least 2.2 billion people have a vision impairment, of whom at least one billion have an impairment that could have been prevented or has yet to be addressed. A systematic review and meta-analysis by Bourne et al. (2017) estimated that 36 million people were blind and 217 million had moderate to severe vision impairment in 2015, with the largest numbers in South and East Asia, and projected substantial increases by 2050 due to population growth and ageing. Vision impairment strongly affects the ability to travel independently, to work, and to take part in community life, which makes safe mobility a central goal of rehabilitation.

Assistive Navigation Technologies

Dakopoulos and Bourbakis (2010) surveyed wearable electronic travel aids for the blind, including ultrasonic canes, laser devices, and camera-based systems. They found that most devices could detect obstacles but were limited by cost, weight, short battery life, and feedback that was difficult to interpret, and that few had been evaluated with blind users in real conditions. Real and Araujo (2019) reviewed navigation systems for the blind and visually impaired and identified obstacle detection, localization, and user interface design as the main open problems, noting that systems should complement, not replace, the white cane. Kuriakose et al. (2022) reviewed tools and technologies for navigation support and highlighted smartphones as promising platforms because they are widely owned and already contain the necessary sensors, while also pointing out that many smartphone solutions depend on internet connectivity and have not been tested with users.

These reviews informed three design decisions in GuideAI: running the core detection offline on the phone, keeping the warnings short and prioritized so they do not overwhelm the user, and designing the application so that it supports the cane rather than replacing it. They also motivated the companion smart cane, which adds ground-level drop detection that a forward-facing camera cannot provide.

Deep Learning Object Detection

Redmon et al. (2016) introduced YOLO (You Only Look Once), which frames object detection as a single regression problem: one pass of a convolutional neural network predicts bounding boxes and class probabilities for the whole image. This made real-time detection possible and started a family of models that has been improved many times. Liu et al. (2016) proposed the Single Shot MultiBox Detector (SSD), another one-stage detector, and Howard et al. (2017) introduced MobileNets, efficient network designs for mobile and embedded vision applications. Together these works established that accurate object detection can run on devices with limited computing power.

Ultralytics YOLO11 (Jocher & Qiu, 2024) is a recent member of the YOLO family. Khanam and Hussain (2024) describe its key architectural changes, including improved feature extraction blocks and attention components, which give better accuracy with fewer parameters than earlier versions. The nano variant, YOLO11n, is small enough to run on a phone. Models of this family are usually first trained on the COCO dataset (Lin et al., 2014), which contains about 330,000 images of everyday scenes labeled with 80 object categories, and are then fine-tuned on task-specific data. GuideAI follows this transfer learning approach: a COCO-pretrained YOLO11n was fine-tuned on Philippine street hazards and vehicles.

Multi-Object Tracking

Detections from single frames tend to flicker: an object may be found in one frame and missed in the next. Bewley et al. (2016) proposed SORT (Simple Online and Realtime Tracking), which links detections across frames by the overlap of their boxes. Zhang et al. (2022) proposed ByteTrack, which improves tracking by also using low-confidence detections to keep existing tracks alive while never starting new tracks from them. GuideAI uses a lightweight tracker in the same spirit: an object must be seen in two

of the last three frames before it is announced, low-confidence detections may keep a known object alive, and growth of an object's box over about one second marks it as approaching.

Vision-Language Models

Large multimodal models can describe images and answer questions about them in natural language. Google's Gemini models accept images and text and can return the locations of objects as bounding boxes (Google AI for Developers, n.d.). Such models recognize a far wider range of objects than a 25-class detector, but they require an internet connection, respond in seconds rather than milliseconds, and can be wrong. For these reasons GuideAI uses Gemini only as an optional online layer for naming unfamiliar objects and for the voice assistant, while all safety warnings come from the offline detector.

Text Recognition, Routing, and Speech

Google ML Kit provides on-device text recognition for mobile applications (Google, n.d.), which allows printed text to be read without an internet connection. Luxen and Vetter (2011) presented the Open Source Routing Machine (OSRM), which computes routes on OpenStreetMap data in real time; GuideAI uses its walking profile to give turn-by-turn directions. Both technologies are free to use, which keeps the application affordable.

Accessibility Standards and Philippine Laws

The Web Content Accessibility Guidelines (WCAG) 2.1 (World Wide Web Consortium, 2018) set out principles for accessible digital content, including sufficient color contrast, text alternatives for non-text content, and operability without precise pointing. GuideAI applies these to a mobile application: every control has a screen reader label, touch targets are large, and every function can be used by voice. In the Philippines, Republic Act No. 7277, the Magna Carta for Disabled Persons (1992), as amended by Republic Act No. 9442 (2007), promotes the rehabilitation, self-development, and integration of persons with disabilities into mainstream society. Republic Act No. 10173, the Data Privacy Act of 2012, governs the processing of personal information; GuideAI

keeps names, contacts, and settings on the phone, has no user accounts, and tells the user when a picture is sent to an online service.

Related Local Studies

[To be completed by the proponents: summarize two to five related capstone projects, theses, or journal articles by Filipino researchers on assistive technology for the visually impaired, obstacle detection, or smart canes, from your university library or Philippine journals. For each, give the author, year, what was developed, the results, and how GuideAI differs. Add each one to the References.]

Synthesis

The reviewed literature shows that (a) vision impairment is widespread and mobility is one of its most limiting effects; (b) electronic travel aids can extend the protection of the white cane, but many are costly, depend on the internet, or are hard to use; (c) one-stage detectors such as YOLO11 make real-time object detection possible on ordinary phones, especially when fine-tuned on task-specific data; (d) tracking across frames reduces flickering and false alarms; and (e) online multimodal models recognize many more objects but are slower and less reliable for safety warnings. GuideAI builds on these findings. Unlike the systems reviewed, it is trained on Philippine hazards and vehicles, keeps its safety functions offline, sends emergency messages through the SIM card, and treats online AI as an optional extra rather than a dependency.

Conceptual Framework

The conceptual framework of the study follows the Input-Process-Output model shown in Table 1.

Table 1. Input-Process-Output conceptual framework of the study

Input	Process	Output
Knowledge requirements: needs of visually impaired pedestrians; common hazards in Dumaguete City; accessibility guidelines; YOLO object detection; Android development. Software requirements: React Native / Expo; YOLO11	Requirements gathering and analysis Dataset collection, merging, and labeling Model training, evaluation, and export to TFLite Iterative design and development	GuideAI: an offline-first Android assistive application for visually impaired pedestrians with real-time obstacle warnings, text reading, walking navigation, voice commands, and emergency SMS Trained YOLO11n model for 25 Philippine street classes

Input	Process	Output
(Ultralytics); TensorFlow Lite; ML Kit; OSRM; Google Gemini API. Hardware requirements: Android phone; laptop with an NVIDIA GPU for training; ESP32 and sensors (planned smart cane). Data: 25,327 labeled images from public datasets and COCO.	of the mobile application (detection, warnings, text reader, navigation, voice, SOS) Unit testing, model testing, and field testing on phones Evaluation with users and IT experts (ISO/IEC 25010)	Smart cane communication protocol

CHAPTER III
METHODOLOGY

This chapter describes the research design, the development process, the system architecture, the dataset and model training, the design of each module, and the methods used to test and evaluate GuideAI.

Research Design

The study used a developmental (applied) research design, which aims to create and validate a product that solves a practical problem. Quantitative methods were used to evaluate the detection model (precision, recall, mAP) and the application's speed, and a descriptive survey based on ISO/IEC 25010 is used to evaluate the quality of the application as perceived by users and IT experts.

Software Development Methodology

The application was built with an iterative, Agile-style prototyping approach. Each cycle produced a working version that was installed on real phones, tested, and improved based on what was observed. New versions of the application code were delivered to the test phones through over-the-air updates (EAS Update), which made it possible to fix problems found in field tests within the same day. The main phases were:

1. Requirements analysis: identifying user needs, common hazards, and accessibility requirements, and deciding that the application must work without login and without internet for its safety functions.
2. Design: defining the architecture, the screens (four tabs and an always-available SOS), and the voice and vibration feedback rules.
3. Data preparation and model training: collecting and merging datasets, training and evaluating the YOLO11n model, and exporting it to TFLite.
4. Development: implementing each module in React Native (Expo) with TypeScript, with native modules for the camera, the neural network, speech, and SMS.
5. Testing: automated unit tests for the logic, accuracy tests for the model, and field tests on phones.

6. Deployment and evaluation: building the installable APK, distributing updates, and evaluating the application with users and experts.

System Architecture

Figure 1 shows the architecture of GuideAI. All modules inside the phone boundary work without an internet connection. The camera passes frames to the YOLO11n model, whose detections go through the tracker and hazard classifier to the hazard announcer, which decides what to say and drives the text-to-speech engine and the vibration motor. The text reader, navigation, voice commands, and emergency SOS are separate modules that share the speech engine and on-device storage. Online services (Google Gemini, OpenStreetMap geocoding and routing) and the planned smart cane are optional.

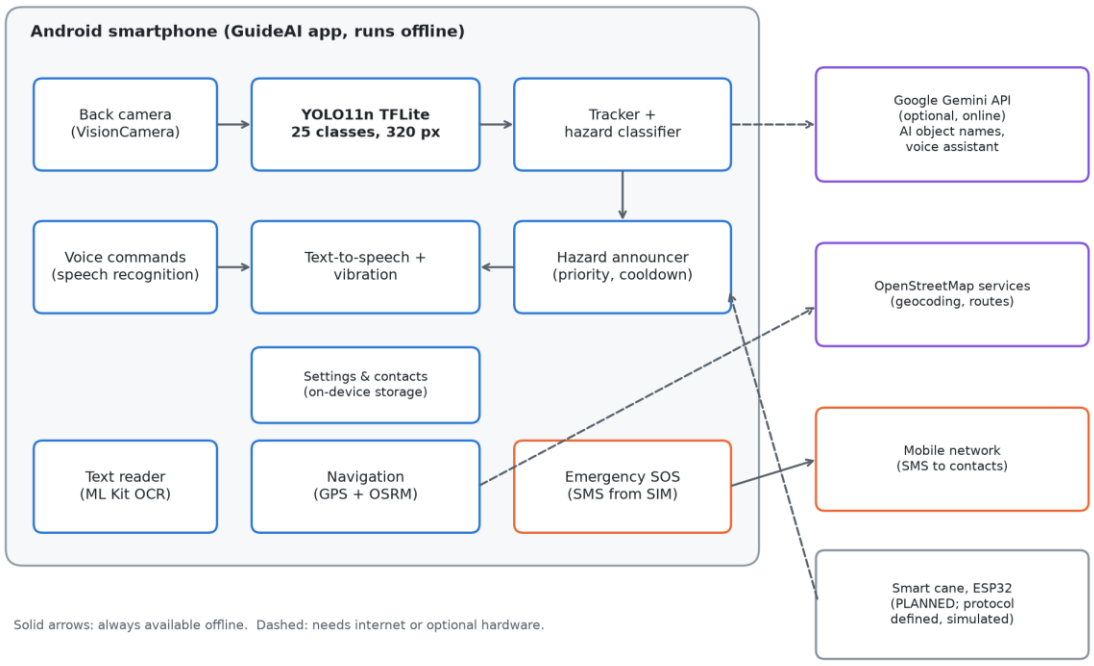


Figure 1. System architecture of GuideAI

System Diagrams

The use-case diagram of GuideAI and its entity-relationship diagram are presented in Appendix G.

Hardware and Software Requirements

Table 2. Hardware used in the study

Component	Specification used in the study
Test phone 1	Honor X9a 5G (RMO-NX1), Android 13
Test phone 2	Realme RMX3171, Android 11
Minimum phone	Android 7.0 or newer with a rear camera (the application is built for Android API level 24+)
Training computer	Laptop with an NVIDIA GeForce RTX 2050 GPU, Windows 11
Planned smart cane	ESP32 microcontroller, RCWL-1601 ultrasonic sensor, VL53L0X laser distance sensor, MPU6050 motion sensor, coin vibration motor, buzzer

Table 3. Software and technologies used

Component	Technology and version
Mobile framework	React Native 0.86 with Expo SDK 57, TypeScript
Camera	react-native-vision-camera 5.2.3 (frame processing on a separate thread)
Neural network runtime	react-native-fast-tflite 3.0.1 (TensorFlow Lite)
Object detection model	Ultralytics YOLO11n, fine-tuned (Ultralytics 8.4, PyTorch 2.6 with CUDA 12.4)
Model conversion	ONNX, then onnx2tf to TensorFlow Lite (float32, 320 x 320 input)
Text recognition	Google ML Kit text recognition (on-device)
Speech	expo-speech (text-to-speech), expo-speech-recognition (speech-to-text, en-PH)
Navigation	Device GPS, Photon and Nominatim geocoding, OSRM walking routes (OpenStreetMap)
Online AI (optional)	Google Gemini API (gemini-3.5-flash-lite)
Emergency SMS	Custom Expo native module using the Android SmsManager
State and storage	Zustand, AsyncStorage, expo-file-system (all on the phone)
Testing	Jest (jest-expo) for automated tests
Build and updates	Expo Application Services (EAS Build for the APK, EAS Update for updates)

Dataset Collection and Preparation

No single public dataset covered the hazards and vehicles of Philippine streets, so nine public datasets were merged into one dataset with a unified list of 25 classes (Table 4). Each source's labels were renamed to the unified class names; labels that should not be learned (for example, closed manhole covers or whole "footpath" regions) were removed while keeping the image as a background example; and large sources were capped so that no single source dominated. Images were resized so that their longer side was at most 416 pixels. Because the street datasets contained no furniture, animals, or indoor scenes, 5,000 images from the COCO 2017 dataset were added for the second training round. Everyday objects in these images (people, vehicles, dogs, cats, chairs, and so on) were labeled automatically by a larger YOLO11m "teacher" model (pseudo-

labeling), and the COCO images also taught the model what ordinary rooms look like, to reduce false alarms such as a laptop screen being called a car.

Table 4. Image sources of the merged dataset

Source (internal name)	Content	Train	Val	Test
ph_vehicles	Philippine road vehicles (tricycles, jeepneys, cars, motorcycles, buses, trucks)	3,500	250	250
ph_tricycle_niluto	Philippine tricycles	225	65	32
open_manhole	Open manholes and drainage openings	2,500	14	18
pothole_ivcnz	Road potholes	1,060	94	89
stairs_rf1	Stairs (boxes)	2,000	250	177
stairs_rf2	Stairs (boxes)	1,384	183	-
stairs_lines	Stairs (stair-edge lines converted to boxes)	2,000	250	-
vip_navigation_bd	Street scenes for visually impaired navigation (poles, barriers, carts, people, vehicles)	4,236	250	250
vip_obstacles	Sidewalk obstacles	1,000	250	-
COCO 2017 (subset)	Everyday indoor and outdoor scenes, pseudo-labeled	4,500	250	250
Total		22,405	1,856	1,066

Note: the original download links and licenses of each public dataset are listed in Appendix F.

Model Training

Training was done on the laptop GPU with the Ultralytics framework. The model started from YOLO11n weights pretrained on COCO and was fine-tuned on the merged dataset with the settings in Table 5. The first model (v1) was trained for 60 epochs and is the model currently installed in the application. A second model (v2) continued from v1 with the COCO images added; it was planned for 45 epochs but the training was interrupted after epoch 37, and its best checkpoint is evaluated in Chapter IV. Figure 2 shows the validation mAP50 of both runs per epoch. The two validation sets differ (v2's includes COCO images), so the curves are not directly comparable; the fair comparison on a common test set is in Chapter IV.

Table 5. Training settings

Setting	Value
Base model	YOLO11n pretrained on COCO (v2: continued from v1)
Input size	320 x 320 pixels
Batch size	32
Epochs	v1: 60; v2: 45 planned, stopped after 37
Optimizer / initial learning rate	MuSGD (Ultralytics automatic choice) / 0.01
Early stopping patience	15 epochs
Augmentation	Ultralytics defaults (mosaic, scaling, translation, HSV color, horizontal flip)
Hardware	NVIDIA GeForce RTX 2050 laptop GPU

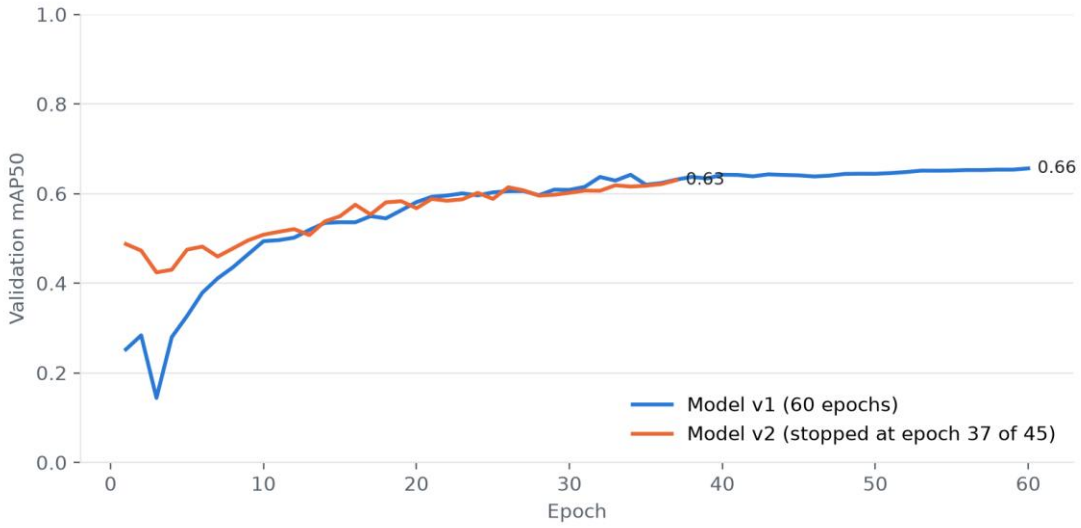


Figure 2. Validation mAP50 per epoch for the two training runs

Model Export and Deployment

The trained PyTorch model was exported to ONNX and converted to TensorFlow Lite (float32) with an input of 320 by 320 pixels, then bundled with the application. On the phone, camera frames are converted on a separate thread into a 320 by 320 "letterboxed" image (scaled without stretching and padded with gray, the same way as in training), so that only a small image crosses into the application code. The model's raw output is decoded into boxes, filtered by a per-class confidence threshold (lower for dangerous classes such as open drainage, potholes, and stairs; higher for people and furniture), and cleaned with class-aware non-maximum suppression.

Obstacle Detection and Warning Pipeline

Figure 3 shows the steps from a camera frame to a spoken warning. About three frames per second are analyzed, which is enough for walking speed and keeps battery use and heat low.

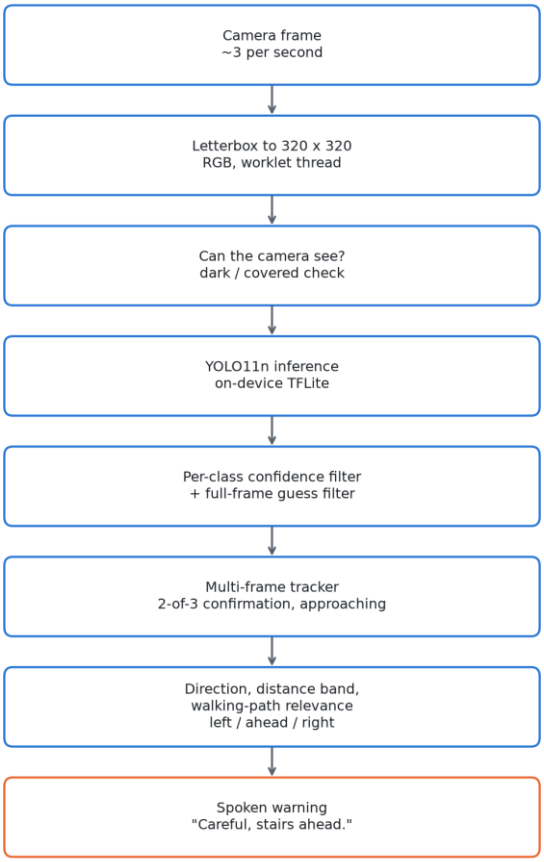


Figure 3. Obstacle detection and warning pipeline

- **View check.** Each frame is checked for darkness and lack of detail. If the camera is covered or in the dark, the application reports nothing and says "Camera view is too dark or covered," so that an empty result is never mistaken for a clear path.
- **Full-frame guess filter.** A low-confidence box that covers nearly the whole picture is ignored, because it is almost always the model guessing at a blank view.
- **Tracking.** An object is announced only after it is seen in two of the last three frames (very confident detections skip the wait). A confirmed object missed for one frame is kept, and boxes are smoothed over frames. If an object's box grows by 15 percent within about 1.2 seconds, it is marked as approaching, and a moving hazard that approaches is raised one danger level.
- **Direction and nearness.** The horizontal position of the box gives the direction (left, ahead, right). An approximate distance is estimated from the box size using typical

real-world heights, or from where the object touches the ground for flat hazards such as potholes, under an assumed camera height of 1.3 m and tilt of 15 degrees. Because this estimate is uncalibrated, it is used only to rank hazards and is never spoken as meters.

- **Walking-path relevance.** Objects off to the side, far away, and not approaching are shown on screen but not spoken, so that the user is not overwhelmed by every parked car or distant person.

Voice Warning Design

Warnings were kept short and consistent so that they can be understood quickly while walking. Stairs, vehicles, people, and animals are named, because knowing what they are changes what the user should do; all other objects are announced as "obstacle". A class is named only if the model is at least 50 percent confident, so that, for example, a tiled floor that slightly resembles stairs is never called "stairs". The rules are summarized in Table 6.

Table 6. Voice warning rules

Detected class	Spoken warning (example)	Voice priority
stairs	"Careful, stairs ahead."	1 (highest)
pothole, open drainage	"Careful, obstacle ahead."	1
car, motorcycle, truck, bus, bicycle, tricycle, jeepney	"Careful, car on your right."	2
person, dog, cat, cow, horse	"Careful, person ahead."	3
chair, bench, pole, barrier, cart, bump, others	"Careful, obstacle on your left."	4

Several hazards seen at the same time are combined into one sentence, most important first, for example "Careful, stairs ahead. Also car on your left." The same warning is not repeated within five seconds unless the hazard becomes more dangerous (higher risk, approaching, or very close). A warning that is being spoken is allowed to finish, unless a hazard with a higher voice priority appears, which then interrupts it. Vibration is given immediately even when the spoken warning has to wait: at the default strength, three strong pulses for critical hazards, two for warnings, and one light pulse for information.

Supporting Modules

Text Reader

The user points the camera at printed text and taps the large Read Text button. Google ML Kit recognizes the text on the phone, and the text is read aloud as natural sentences. When online, the user can also ask for a description of the scene, which is answered by Google Gemini.

Walking Navigation

The user chooses or speaks a destination. Common places in Dumaguete City are stored in the application; other places are found with the Photon and Nominatim geocoders. A walking route is computed with OSRM, and spoken directions are given with the live distance to each turn, a heads-up 30 meters before the turn, and automatic rerouting when the user leaves the route. The user is asked to confirm before starting a walk longer than 5 kilometers. "Where am I?" speaks the current street and area.

Voice Commands and Assistant

A large Voice Command button starts speech recognition (Philippine English). Commands are matched offline against English and Tagalog keywords (for example "navigate to the market", "nasaan ako", "basahin", "tulong"). When online, free-form requests in English, Tagalog, or Cebuano are understood by Google Gemini, which chooses the action and gives a short spoken reply. Emergency and stop commands are always handled offline and never depend on the online assistant.

Emergency SOS

The SOS button is always visible on the home screen. Pressing it starts a five-second spoken countdown that can be cancelled. When the countdown ends, the application gets the GPS location and sends this message directly from the SIM card to every saved emergency contact: "EMERGENCY: [name] needs help near [area]. Location: https://maps.google.com/?q=[latitude],[longitude] - sent by GuideAI, an app for the visually impaired." The message uses plain characters so that it fits in a single SMS. Philippine emergency hotlines (911, 117, 143, 160) are listed for one-tap calling.

AI Object Names (Optional, Online)

Every 6 to 20 seconds while the camera is active and the phone is online, one sharper picture (512 pixels) is sent to Google Gemini, which returns the names and boxes of the objects it sees. A name that matches a tracked detection refines its label on screen (for example, "chair" with "AI: monobloc chair"); objects the offline model has no class for are shown with dashed boxes. AI names are marked as such and shown without a percentage, because the AI's certainty is not a calibrated score. The online layer never triggers or silences a safety warning, it pauses while the camera cannot see, and it can be turned off in Settings.

Teach My AI

When turned on, the application saves pictures in which the model was unsure, plus one picture a minute to catch missed objects, and shows a Teach button that saves the current view with a name typed by a helper. The pictures stay on the phone until copied to a computer, where a script imports them for checking in a labeling tool, validates the labels, and adds them to the training set, keeping pictures taken within the same few minutes in the same split so that near-identical frames do not inflate the test results. Nothing is retrained automatically.

Smart Cane Protocol

The companion cane is defined as a Bluetooth Low Energy peripheral that sends front (ultrasonic) and ground (laser) distances about ten times per second, plus button and fall events (Appendix C). The application learns the normal ground distance while walking and warns of a drop when the ground is 25 cm farther than normal for two readings in a row, of a step or curb when it is 20 cm closer, and of an obstacle at chest height within 150 cm (stop within 70 cm). The cane must also warn on its own with its vibration motor and buzzer if the phone is unavailable.

Accessibility Design

- The application is organized into four tabs (Home, Detect, Navigate, Read) with large buttons and an always-visible SOS button.

- Every control has a TalkBack label and hint, touch targets are at least 48 by 48 density-independent pixels, and the dark theme uses high-contrast text.
- Each screen announces itself when opened; obstacle detection starts automatically when the Detect tab is opened, and tapping anywhere on the camera view pauses or resumes it.
- Settings are saved immediately, with no Save button to find; speech speed, voice accent (Philippine English by default), and vibration strength are adjustable.
- There is no login and no account; the screen stays awake during detection so that warnings continue.

Testing and Evaluation

Automated Software Testing

The logic of the application (detection decoding, tracking, distance bands, warning rules, voice command parsing, navigation formatting, contact validation, SMS text, the AI identification merger, and others) was covered by automated unit tests written with Jest and run after every change.

Model Evaluation

The models were evaluated on the held-out test split of 1,066 images that were not used in training. The following standard metrics were computed, where TP, FP, and FN are true positives, false positives, and false negatives at an intersection-over-union (IoU) threshold:

- Precision = TP / (TP + FP)
- Recall = TP / (TP + FN)
- AP: the area under the precision-recall curve for one class; mAP50 is the mean AP over classes at IoU 0.50, and mAP50-95 is the mean over IoU thresholds from 0.50 to 0.95 in steps of 0.05.

On-Device Performance and Field Testing

The frames analyzed per second and the time the model takes per frame were measured inside the application from real timestamps and shown on the detection screen.

Field tests were performed on the test phones by pointing the camera at people, furniture, laptops, classrooms, and covered views, while screenshots were captured over USB.

User Acceptance Evaluation

The quality of the application is evaluated with a questionnaire based on the ISO/IEC 25010 quality model (Appendix A), covering functional suitability, performance efficiency, usability, reliability, and security. For visually impaired respondents, the questions are read aloud and answers are recorded by a facilitator. Each item is rated on a five-point Likert scale, and the weighted mean of each characteristic is interpreted with the scale in Table 7.

Table 7. Likert scale and interpretation

Scale	Range of weighted mean	Verbal interpretation
5	4.21 - 5.00	Strongly Agree (Excellent)
4	3.41 - 4.20	Agree (Very Good)
3	2.61 - 3.40	Neutral (Good)
2	1.81 - 2.60	Disagree (Fair)
1	1.00 - 1.80	Strongly Disagree (Poor)

Weighted mean = $\Sigma(f \times w) / N$, where f is the number of respondents who chose a rating, w is the rating (1 to 5), and N is the total number of respondents. Respondents: [number] visually impaired participants from [organization], selected by purposive sampling, and [number] IT experts.

Ethical Considerations

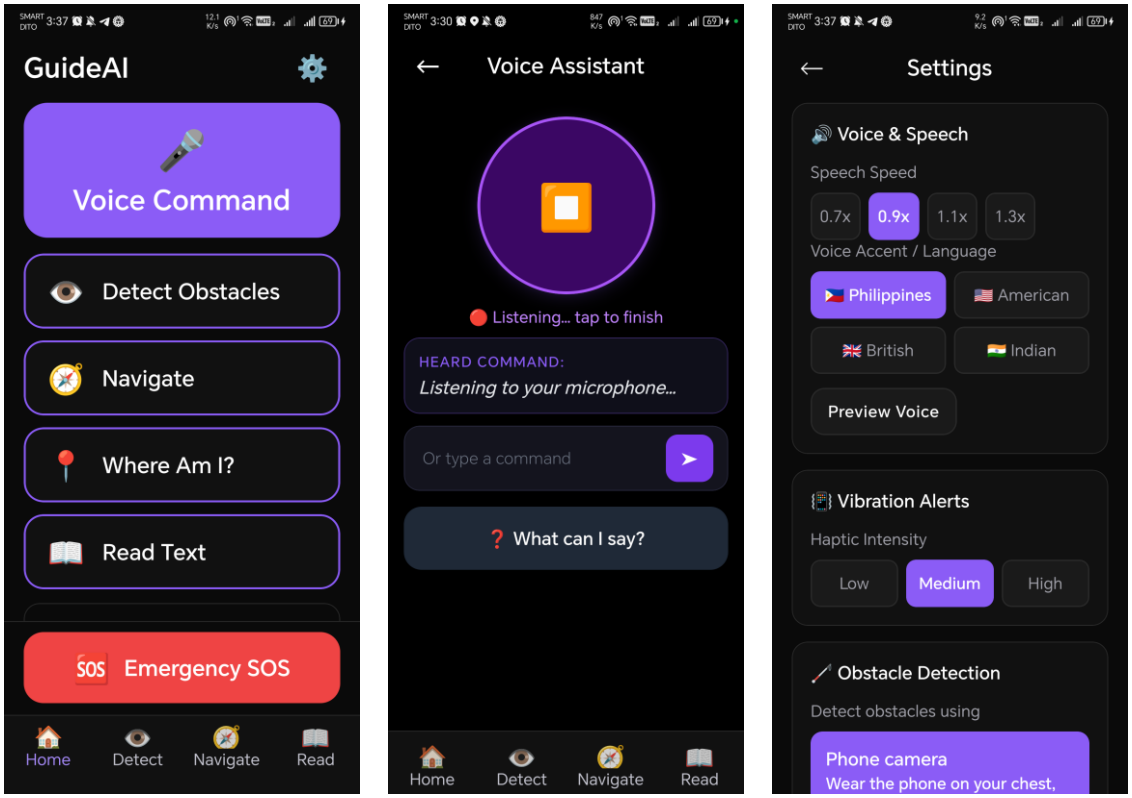
- Participants will take part voluntarily after informed consent is explained verbally, and may stop at any time. Field tests with blind participants are done in safe areas with a sighted companion.
- The application stores the user's name, emergency contacts, and settings only on the phone. There are no accounts and no central database, in line with the Data Privacy Act of 2012.
- When the optional online features are used, a camera picture or the spoken request is sent to Google Gemini; this is disclosed in Settings and can be turned off.
- The application states that it is an assistive aid and does not guarantee safety; users are reminded to keep using their cane.

CHAPTER IV
RESULTS AND DISCUSSION

This chapter presents the developed system and the results of the model evaluation, on-device performance measurements, software and field tests, and user evaluation, organized according to the specific problems in Chapter I.

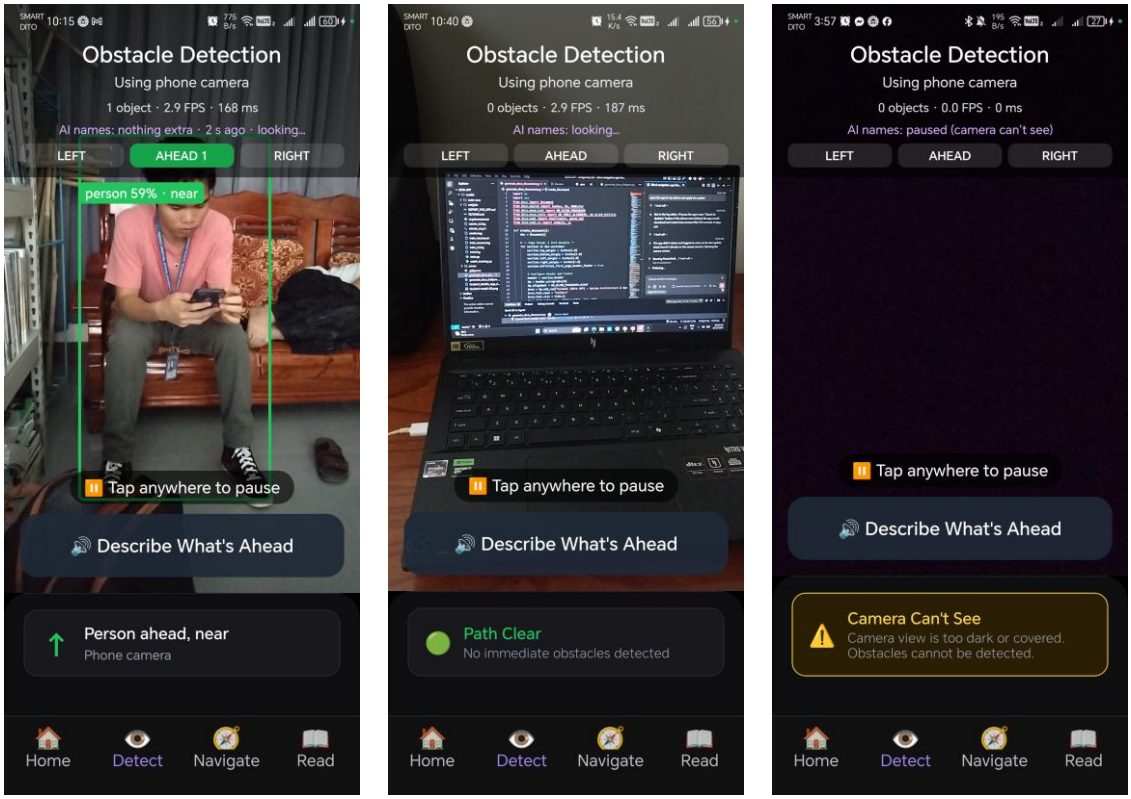
The Developed System

GuideAI was developed and installed on two Android phones as a release APK, with later changes delivered through over-the-air updates. Figures 4 to 6 show its main screens as captured from the test phone.



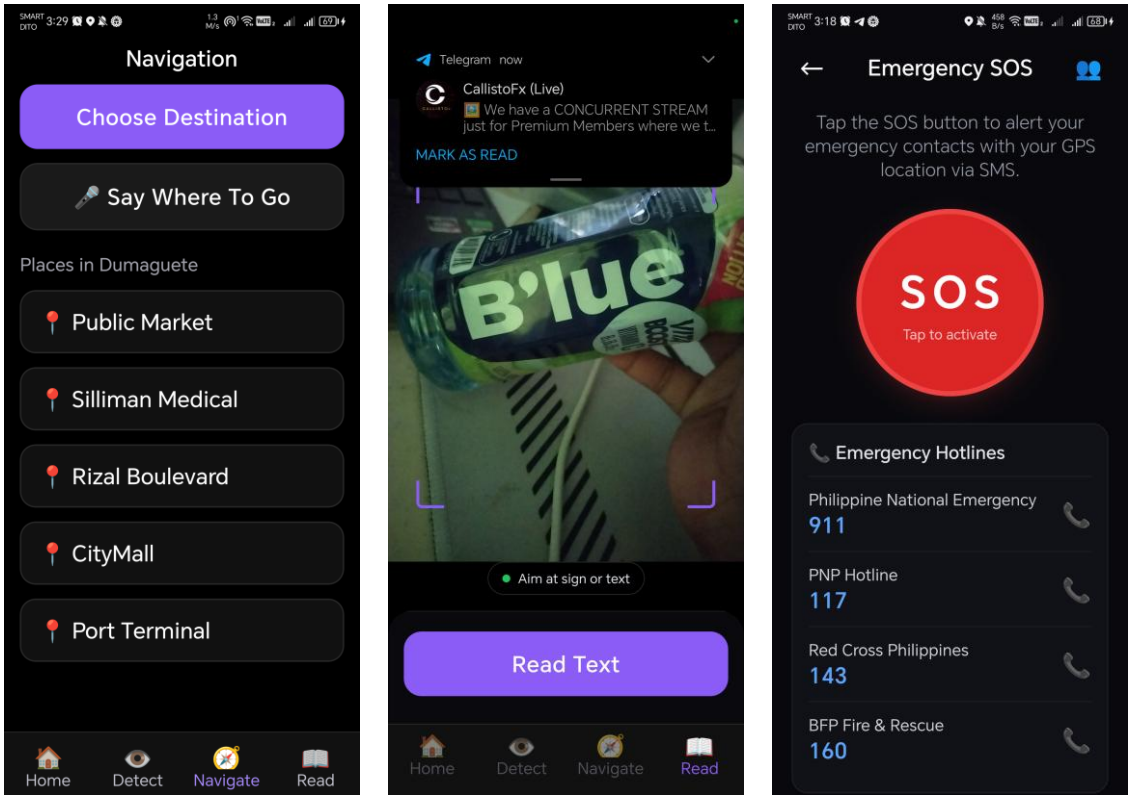
Home *Voice command* *Settings*

Figure 4. Home, voice command, and settings screens



Person detected ahead *Laptop: no false alarm* *Camera covered*

Figure 5. Obstacle detection screen in three situations (field test, 17 October 2026)



Choose destination *Text reader* *Emergency SOS*

Figure 6. Navigation, text reader, and emergency SOS screens

The detection screen (Figure 5) shows the live camera view with a box and label for each detected object, including the model's confidence and the nearness band, a status strip with the number of objects and the measured speed, and left/ahead/right indicators colored by the most serious hazard on each side. These visual elements are for sighted helpers and testers; the blind user receives the same information through speech and vibration.

Model Performance

Table 8 compares the two trained models on the same held-out test set of 1,066 images with 3,451 labeled objects. The retrained model (v2) found considerably more of the objects (recall 0.477 versus 0.349) and achieved a higher mAP50 (0.480 versus 0.386) and mAP50-95 (0.328 versus 0.270). Its precision was lower (0.536 versus 0.697), which is expected: v1 rarely predicted the classes it had no training data for, so the few predictions it made were more often correct, while v2 now attempts these classes. On the training laptop's GPU, both models took about 2.2 to 2.4 milliseconds per image.

Table 8. Overall detection accuracy on the test set (1,066 images)

Model	Precision	Recall	mAP50	mAP50-95
v1 (installed in the application)	0.697	0.349	0.386	0.270
v2 (retrained with COCO images, epoch 37 of 45)	0.536	0.477	0.480	0.328

Table 9. Per-class test results, sorted by v2 mAP50

Class	v1 mAP50	v2 precision	v2 recall	v2 mAP50
bus	0.814	0.756	0.945	0.929
tricycle	0.914	0.869	0.879	0.909
jeepney	0.823	0.799	0.852	0.880
person	0.780	0.693	0.822	0.842
car	0.814	0.686	0.842	0.827
cart	0.773	0.583	0.789	0.804
stairs	0.684	0.597	0.789	0.740
motorcycle	0.701	0.681	0.774	0.731
truck	0.665	0.649	0.620	0.683
pole	0.595	0.619	0.644	0.653
pothole	0.508	0.561	0.485	0.518
dog	0.205	0.430	0.400	0.492
bicycle	0.277	0.451	0.538	0.481
horse	0.000	0.309	0.417	0.364
open_drainage	0.327	0.742	0.278	0.326
potted plant	0.085	0.424	0.400	0.322
traffic light	0.000	0.239	0.201	0.275
barrier	0.209	0.623	0.222	0.228
cow	0.000	0.567	0.133	0.191
cat	0.020	0.151	0.222	0.190
bench	0.028	0.273	0.133	0.080
chair	0.030	0.170	0.061	0.050
fire hydrant	0.000	0.000	0.000	0.006
stop sign	0.000	1.000	0.000	0.000

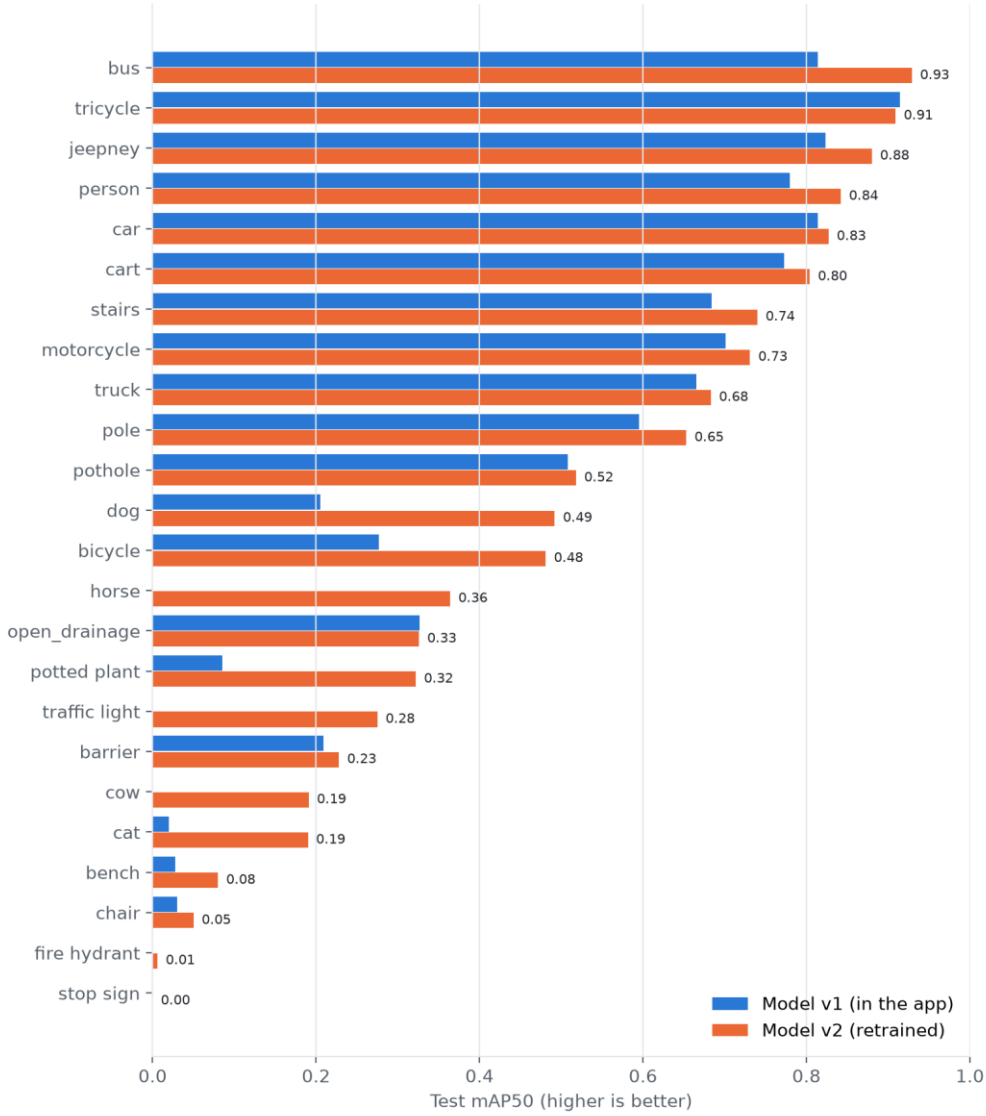


Figure 7. Per-class test mAP50 of the two models

The per-class results (Table 9 and Figure 7) show three groups. The model is strong on Philippine vehicles and people: tricycles, buses, jeepneys, people, cars, and carts reach an mAP50 of 0.80 to 0.93, and motorcycles and trucks around 0.68 to 0.73. It is moderate on stairs (0.74), poles (0.65), and potholes (0.52), which are among the most important hazards. It is weak on open drainage (0.33) and barriers (0.23), and on the classes that had no labeled examples in the street datasets: chairs, benches, cats, cows, fire hydrants, and stop signs (0.00 to 0.19). v1 had essentially no ability to detect these, and the COCO images added in v2 raised dogs to 0.49, horses to 0.36, and potted plants

to 0.32, but furniture remains poor. This matches what was seen in field tests, where a classroom full of chairs and tables produced no detections except the nearest person.

Two cautions apply to these numbers. First, the 250 COCO test images were labeled by the YOLO11m teacher model, not by people, so results for everyday classes are measured against automatic labels. Second, open drainage had only 18 test images and fire hydrants, stop signs, and some animals had very few, so their results are uncertain. More hand-labeled local images, especially of open drainage, barriers, furniture, and stairs in Dumaguete, are the most direct way to improve accuracy.

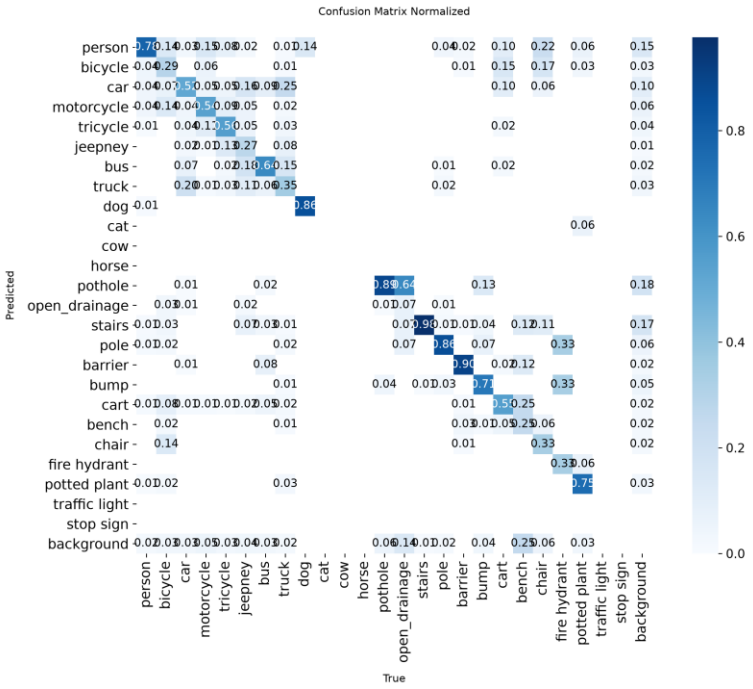


Figure 8. Normalized confusion matrix of model v1 on the validation set

On-Device Performance

Table 10 summarizes the speed measured inside the application on the Honor X9a 5G during the field tests. The application deliberately analyzes a frame about every 350 milliseconds, so the analysis rate of 2.5 to 3.0 frames per second is close to its designed maximum, with model time well below the interval. The online object naming answered in 1.6 to 2.4 seconds when tested on sample photos after reducing the model's reasoning setting (it took 9 to 14 seconds with the default setting); it runs alongside the offline detection and never delays the warnings.

Table 10. On-device performance (Honor X9a 5G, Android 13)

Measure	Result
Frames analyzed per second (measured in the app)	2.5 - 3.0
Model time per frame on the phone (measured in the app)	135 - 215 ms
Model time per image on the training GPU (RTX 2050)	2.2 - 2.4 ms
Online object naming (Gemini), per picture	1.6 - 2.4 s (sample photos, PC test)
Model file size	TFLite float32, 320 x 320 input

Software Testing

All 236 automated tests in 15 test suites passed on the final version of the application. Table 11 lists the main areas covered.

Table 11. Areas covered by the automated tests

Area	Examples of what is tested
Detection decoding	YOLO output decoding, letterbox coordinates, non-maximum suppression, label sets
Tracking and risk	two-of-three confirmation, approaching detection, low-confidence rescue, box smoothing, walking-path relevance
Distance and view	nearness bands, camera dark/covered check, full-frame guess filter
Voice warnings	class-to-speech rules, voice priority, combining, repeat suppression, interruptions
Voice commands	English/Tagalog/Cebuano intent matching, Gemini reply validation, offline fallback
Navigation	route step formatting, distance and duration wording, off-route distance
Emergency	Philippine phone number validation, SMS message format
AI identification	parsing Gemini boxes, matching to YOLO objects, scene-change handling, JPEG encoding
Teach My AI	picture saving rules, BMP encoding, suggestion files

Field Testing on the Phone

Field tests on the phone revealed problems that the automated tests could not, which were fixed and re-tested. Table 12 lists the test cases and observed results.

Table 12. Field test cases and results

Test case	Expected	Observed result
Person in front of the camera	Person detected; warning spoken	Detected ("person 59% · near", ahead); warning spoken
Laptop screen filling the view	No false vehicle	Earlier test (13 Oct): v1 called a laptop a "car". 17 Oct, v1 with the full-frame filter: 0 objects, "Path Clear"
Phone lying face down	"Camera can't see", no hazard	First test: false "Open drain ahead, very close" for 40+ s. Fixed: no detections while the view is dark; re-test showed 0 objects and the warning
Classroom with chairs, tables, students	Obstacles detected	Only the nearest person detected; chairs and tables missed (model limitation)
Moving the camera to a new	Old AI boxes removed	First test: AI boxes stayed over a blank wall. Fixed

Test case	Expected	Observed result
view		with scene-change detection; re-scan in about 1.5 s
AI object naming on sample photos	Correct names and boxes	Tricycles, motorcycle, cars, pothole, stairs boxed correctly (Figure 8)
Text reader on a printed label	Text read aloud	[To be recorded during the evaluation]
Navigation to a stored place	Route and spoken steps	Route found and steps shown on screen; spoken guidance while walking: [to be recorded]
Emergency SOS	Countdown, SMS sent	Countdown and screen verified; [real SMS delivery test: to be recorded]



Figure 9. Objects found and named by the online AI layer on sample test photos

User Acceptance Evaluation

[To be completed after the evaluation. Present the respondents, then the weighted mean and interpretation for each ISO/IEC 25010 characteristic in Table 13, followed by a discussion of the comments of the respondents.]

Table 13. Summary of the ISO/IEC 25010 evaluation (to be filled in)

Quality characteristic	Visually impaired users (mean)	IT experts (mean)	Interpretation
Functional suitability			
Performance efficiency			
Usability			
Reliability			
Security			
Overall			

Discussion

The results answer the problems of the study as follows. Hazards relevant to Philippine streets, including tricycles, jeepneys, stairs, potholes, poles, and open drainage, can be detected by a phone camera with an on-device model (Problems 1 and 2), with good accuracy for vehicles and people and moderate accuracy for stairs and potholes. The model runs offline on a mid-range phone at about three analyses per second (Problem 5), which is sufficient for walking pace. Short, named, prioritized warnings with vibration, confirmed across frames and limited to objects in the walking path, communicate hazards without constant speech (Problem 3). The text reader, navigation, voice commands, and SIM-based SOS complete the application and are fully operable by voice and screen reader (Problem 4).

The field tests showed the value of testing on real devices: a covered camera produced a confident-sounding false alarm, which would teach a blind user to ignore real warnings, and it was corrected by treating a dark view as "cannot see" rather than as a scene to analyze. The main limitations are the model's weakness on furniture, barriers, and open drainage due to limited training data, the uncalibrated distance estimate, dependence on lighting, and the smart cane not yet being built.

CHAPTER V
SUMMARY, CONCLUSIONS, AND RECOMMENDATIONS

Summary of Findings

1. A merged dataset of 25,327 images with 25 classes was prepared from nine public datasets of Philippine vehicles, potholes, open drainage, stairs, and sidewalk obstacles, plus 5,000 pseudo-labeled COCO images.
2. A YOLO11n model was fine-tuned and deployed on Android phones as a TensorFlow Lite model that runs without internet. On the test set, the installed model (v1) reached an mAP50 of 0.386, and the retrained model (v2) reached 0.480 with a recall of 0.477.
3. The model performs best on tricycles, buses, jeepneys, people, cars, and carts (mAP50 0.80 to 0.93), moderately on stairs, poles, and potholes (0.52 to 0.74), and poorly on furniture, barriers, open drainage, and rare classes.
4. On a mid-range phone, the application analyzed 2.5 to 3.0 frames per second with 135 to 215 ms of model time per frame.
5. The warning system confirms detections over frames, ignores views the camera cannot see, names stairs, vehicles, people, and animals, calls other objects "obstacle", and orders warnings by danger.
6. The text reader, walking navigation, bilingual voice commands, optional online assistant and object naming, Teach My AI, and SIM-based emergency SOS were implemented; all 236 automated tests passed.
7. Field tests found and led to the correction of a false open-drain alarm when the camera was covered and of outdated AI boxes after the camera moved.
8. User acceptance results: [to be summarized after the evaluation].

Conclusions

Based on the findings, the following conclusions are drawn:

1. An ordinary mid-range Android phone can detect common street hazards and vehicles in real time without an internet connection, making an affordable assistive aid feasible for visually impaired pedestrians in the Philippines.

2. Training on local data matters: the model recognizes tricycles and jeepneys well because they were in the training data, while classes without local or labeled data remain weak.
3. How warnings are delivered is as important as detection accuracy: confirmation across frames, a "cannot see" state, walking-path filtering, and short prioritized messages are necessary to avoid false or overwhelming alerts.
4. Keeping safety functions offline and using online AI only as an optional extra gives the benefits of both without making safety depend on the network.
5. Testing with real phones in real conditions is essential, because some serious problems appear only there.
6. [Conclusion on user acceptance, after the evaluation.]

Recommendations

1. Collect and hand-label photos of Dumaguete sidewalks, especially open drainage, barriers, curbs, stairs (going up and down), and furniture, using the Teach My AI feature, and retrain the model.
2. Complete the training of model v2 (or a successor), evaluate it against v1 on the same test set, and install it in the application if it is better.
3. Build the ESP32 smart cane according to the defined protocol to add ground-level drop and chest-height detection that a camera cannot provide.
4. Calibrate the distance estimate outdoors with measured distances, or use phones with depth sensors, before ever speaking distances in meters.
5. Conduct longer field trials with visually impaired users together with orientation and mobility instructors, in different lighting and weather conditions.
6. Convert the model to an 8-bit quantized version to increase speed and reduce battery use, and test on lower-end phones.
7. Add more regional languages for voice commands and replies, such as Hiligaynon and Ilocano.

REFERENCES

Bewley, A., Ge, Z., Ott, L., Ramos, F., & Upcroft, B. (2016). Simple online and realtime tracking. In 2016 IEEE International Conference on Image Processing (ICIP) (pp. 3464-3468). IEEE.

Bourne, R. R. A., Flaxman, S. R., Braithwaite, T., Cicinelli, M. V., Das, A., Jonas, J. B., et al. (2017). Magnitude, temporal trends, and projections of the global prevalence of blindness and distance and near vision impairment: A systematic review and meta-analysis. The Lancet Global Health, 5(9), e888-e897.

Dakopoulos, D., & Bourbakis, N. G. (2010). Wearable obstacle avoidance electronic travel aids for blind: A survey. IEEE Transactions on Systems, Man, and Cybernetics, Part C (Applications and Reviews), 40(1), 25-35.

Google. (n.d.). Text recognition v2. ML Kit. https://developers.google.com/ml-kit/vision/text-recognition/v2

Google AI for Developers. (n.d.). Image understanding: Object detection. Gemini API documentation. https://ai.google.dev/gemini-api/docs/image-understanding

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). MobileNets: Efficient convolutional neural networks for mobile vision applications. arXiv:1704.04861.

ISO/IEC. (2011). ISO/IEC 25010:2011 Systems and software engineering - Systems and software Quality Requirements and Evaluation (SQuaRE) - System and software quality models. International Organization for Standardization.

Jocher, G., & Qiu, J. (2024). Ultralytics YOLO11 (Version 11.0.0) [Computer software]. https://github.com/ultralytics/ultralytics

Khanam, R., & Hussain, M. (2024). YOLOv11: An overview of the key architectural enhancements. arXiv:2410.17725.

Kuriakose, B., Shrestha, R., & Sandnes, F. E. (2022). Tools and technologies for blind and visually impaired navigation support: A review. IETE Technical Review, 39(1), 3-18.

Lin, T.-Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., Dollár, P., & Zitnick, C. L. (2014). Microsoft COCO: Common objects in context. In European Conference on Computer Vision (ECCV) (pp. 740-755). Springer.

Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C.-Y., & Berg, A. C. (2016). SSD: Single shot multibox detector. In European Conference on Computer Vision (ECCV) (pp. 21-37). Springer.

Luxen, D., & Vetter, C. (2011). Real-time routing with OpenStreetMap data. In Proceedings of the 19th ACM SIGSPATIAL International Conference on Advances in Geographic Information Systems (pp. 513-516). ACM.

Real, S., & Araujo, A. (2019). Navigation systems for the blind and visually impaired: Past work, challenges, and open problems. Sensors, 19(15), 3404.

Redmon, J., Divvala, S., Girshick, R., & Farhadi, A. (2016). You only look once: Unified, real-time object detection. In Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR) (pp. 779-788).

Republic Act No. 7277. (1992). Magna Carta for Disabled Persons. Republic of the Philippines.

Republic Act No. 9442. (2007). An Act Amending Republic Act No. 7277. Republic of the Philippines.

Republic Act No. 10173. (2012). Data Privacy Act of 2012. Republic of the Philippines.

World Health Organization. (2019). World report on vision. World Health Organization.

World Wide Web Consortium. (2018). Web Content Accessibility Guidelines (WCAG) 2.1. https://www.w3.org/TR/WCAG21/

Zhang, Y., Sun, P., Jiang, Y., Yu, D., Weng, F., Yuan, Z., Luo, P., Liu, W., & Wang, X. (2022). ByteTrack: Multi-object tracking by associating every detection box. In European Conference on Computer Vision (ECCV) (pp. 1-21). Springer.

[Add the local studies reviewed in Chapter II and the dataset sources in Appendix F.]

APPENDIX A: EVALUATION QUESTIONNAIRE (ISO/IEC 25010)

Instructions: Please rate each statement about GuideAI. 5 - Strongly Agree, 4 - Agree, 3 - Neutral, 2 - Disagree, 1 - Strongly Disagree. For visually impaired respondents, the facilitator reads each statement aloud and records the answer.

	Functional suitability	**5**	**4**	**3**	**2**	**1**
1	The application warns me about obstacles and vehicles in front of me.					
2	The warnings tell me the direction of the obstacle (left, ahead, right).					
3	The text reader reads printed text correctly.					
4	Walking navigation guides me to my destination.					
5	The emergency SOS sends my location to my contacts.					

	Performance efficiency	**5**	**4**	**3**	**2**	**1**
1	Warnings come quickly enough while I walk.					
2	The application works without an internet connection.					
3	The application does not drain the battery too quickly.					

	Usability	**5**	**4**	**3**	**2**	**1**
1	I can use the application without seeing the screen.					
2	The spoken warnings are short and easy to understand.					
3	The buttons are easy to find with TalkBack or by touch.					
4	The voice commands understand what I say.					
5	I could learn to use the application quickly.					

	Reliability	**5**	**4**	**3**	**2**	**1**
1	The application does not crash or freeze.					
2	The application rarely gives false warnings.					
3	The application tells me when the camera cannot see.					

	Security	**5**	**4**	**3**	**2**	**1**
1	I am comfortable that my contacts and name stay on my phone.					
2	The application tells me when a picture is sent online.					

Comments and suggestions: _____

APPENDIX B: VOICE COMMANDS

Table 14. Voice command examples

Action	English (offline)	Tagalog (offline)
Navigate	"Navigate to the public market", "Take me to Silliman Medical"	"Pumunta sa palengke", "Dalhin mo ako sa ospital"
Where am I	"Where am I?", "What street is this?"	"Nasaan ako?"
Read text	"Read text", "What does this say?"	"Basahin mo ito"
Detect obstacles	"Detect obstacles", "What is around me?"	"Ano ang nasa harap ko?"
Describe scene (online)	"What is in front of me?", "Describe this"	(online assistant)
Emergency	"Emergency", "Help", "Send SOS"	"Tulong", "Saklolo"
Controls	"Stop", "Repeat", "Mute", "Settings"	"Tigil", "Ulitin"

When the phone is online, free-form requests in English, Tagalog, or Cebuano (for example "Asa ko?", "Tabang") are understood by the online assistant; emergency and stop commands are always handled offline.

APPENDIX C: SMART CANE BLUETOOTH PROTOCOL (SUMMARY)

The ESP32 cane is a Bluetooth Low Energy peripheral advertising as "GuideAI-Cane". The full specification is in hardware/PROTOCOL.md of the project.

Table 15. Cane service 6e7a0001-4f1b-4c1d-9b3a-2a6e8c9d0b00

Characteristic	UUID	Property	Content
Sensor readings	6e7a0002-4f1b-4c1d-9b3a-2a6e8c9d0b00	Notify, ~10 Hz	4 bytes: frontCm (uint16), groundCm (uint16), little-endian; 0xFFFF = no reading
Events	6e7a0003-4f1b-4c1d-9b3a-2a6e8c9d0b00	Notify	0x01 SOS held 2 s; 0x02 mode short press (Where am I); 0x03 mode long press (voice); 0x04 fall detected
Commands	6e7a0004-4f1b-4c1d-9b3a-2a6e8c9d0b00	Write	0x01 + pattern: vibrate; 0x02 + duration: buzzer
Battery level	0x2A19 (service 0x180F)	Read, Notify	0-100 %

Table 16. Cane hazard thresholds (shared by the firmware and the application)

Rule	Value
Front obstacle: warning / stop	≤ 150 cm / ≤ 70 cm
Normal ground distance (start value, then learned)	80 cm
Drop (stairs down, canal, hole)	ground ≥ 25 cm farther than normal, 2 readings in a row
Step or curb	ground ≥ 20 cm closer than normal

APPENDIX D: USER MANUAL

Installing

1. Install the GuideAI APK on an Android phone (allow installation from this source when asked).
2. Open GuideAI and allow access to the camera, microphone, location, and SMS when asked.
3. Enter your name and add at least one emergency contact (Settings > Emergency Contacts).

Detecting Obstacles

1. Open the Detect tab, or say "Detect obstacles". Detection starts automatically.
2. Hold or wear the phone at chest height with the back camera facing forward.
3. Listen for warnings such as "Careful, stairs ahead." Tap anywhere on the screen to pause or resume.
4. Tap "Describe What's Ahead" to hear everything the camera currently sees.
5. If you hear "Camera view is too dark or covered," uncover the camera or find more light.

Reading Text, Navigating, and Getting Help

- Read tab: point the camera at the text about 30 cm away and tap Read Text.
- Navigate tab: choose a place or tap "Say Where To Go", then follow the spoken directions.
- "Where Am I?": hear your current street and area.
- Emergency SOS: press the red button; after the 5-second countdown, your location is texted to your contacts. Tap again during the countdown to cancel.

Settings

- Speech speed, voice accent, and vibration strength.
- Detection source: phone camera, cane, or both.
- AI object names (online) on or off; Teach My AI for collecting training pictures.
- Check for Updates downloads the newest version of the application.

APPENDIX E: DETECTION CLASSES AND SPOKEN WORDS

Table 17. The 25 detection classes

Model class	Shown on screen	Spoken (+ ahead / on your left / on your right)
person	Person	Careful, person ...
bicycle	Bicycle	Careful, bicycle ...
car	Car	Careful, car ...
motorcycle	Motorcycle	Careful, motorcycle ...
tricycle	Tricycle	Careful, tricycle ...
jeepney	Jeepney	Careful, jeepney ...
bus	Bus	Careful, bus ...
truck	Truck	Careful, truck ...
dog	Dog	Careful, dog ...
cat	Cat	Careful, cat ...
cow	Large animal	Careful, cow ...
horse	Horse	Careful, horse ...
pothole	Pothole	Careful, obstacle ...
open_drainage	Open drain	Careful, obstacle ...
stairs	Stairs	Careful, stairs ...
pole	Pole	Careful, obstacle ...
barrier	Barrier	Careful, obstacle ...
bump	Bump	Careful, obstacle ...
cart	Cart	Careful, obstacle ...
bench	Bench	Careful, obstacle ...
chair	Chair	Careful, obstacle ...
fire hydrant	Fire hydrant	Careful, obstacle ...
potted plant	Plant pot	Careful, obstacle ...
traffic light	Traffic light	Careful, obstacle ...
stop sign	Stop sign	Careful, obstacle ...

APPENDIX F: DATASET SOURCES

[For each public dataset in Table 4, give its full title, author or organization, download URL (for example the Roboflow Universe or Kaggle page), license, and the date it was downloaded. COCO 2017: Lin et al. (2014), https://cocodataset.org, Creative Commons Attribution 4.0 for annotations.]

APPENDIX G: SYSTEM USE CASES AND DATA MODEL

The use-case diagram shows the functions each actor can perform. The blind pedestrian is the primary user. Family members or helpers receive emergency messages and can mark an SOS as resolved. Google Gemini and OpenStreetMap are online services used only for optional features. The smart cane is shown as planned, because its protocol is defined but its hardware was not built in this study. The table below describes the main use cases.

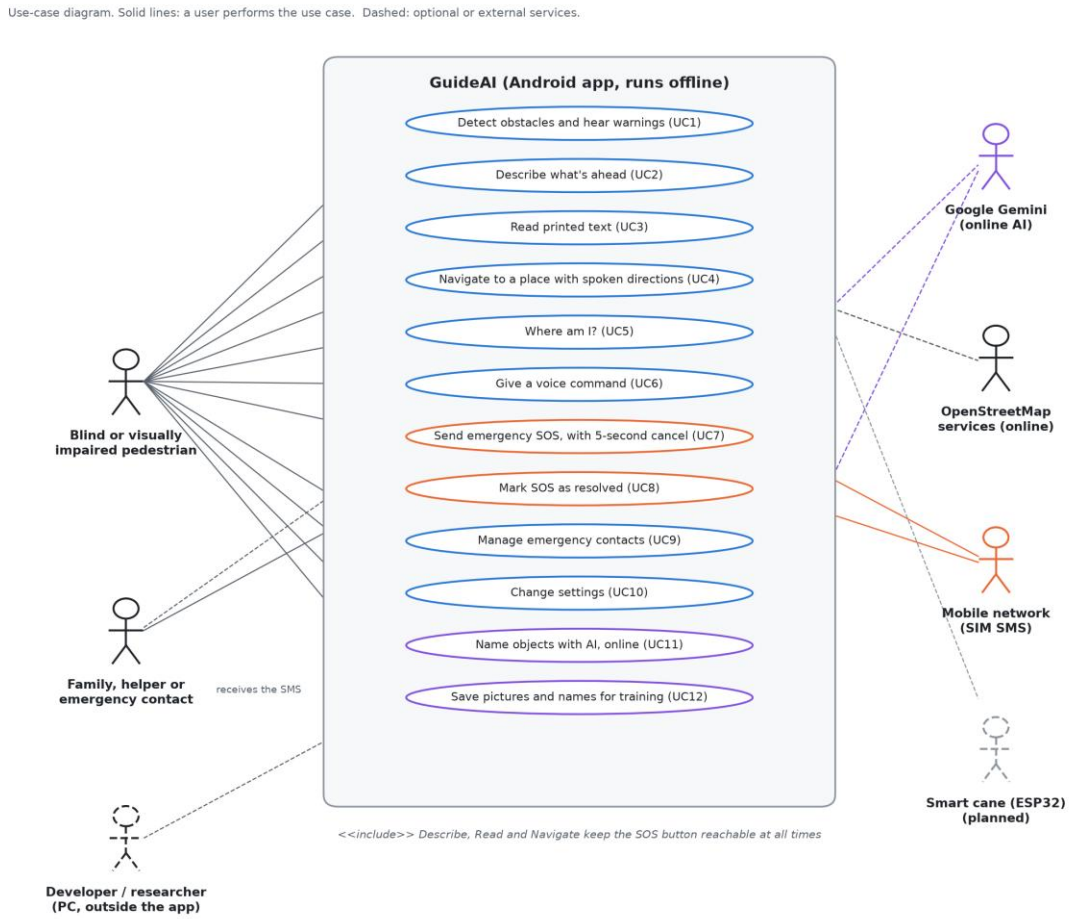


Figure 10. Use-case diagram of GuideAI

Table 18. Description of the main use cases

Use case	Actor	Main flow
UC1 Detect obstacles and hear warnings	Pedestrian	Camera analyzes frames; confirmed hazards are spoken and vibrate
UC2 Describe what's ahead	Pedestrian	All current warnings are spoken in priority order
UC3 Read printed text	Pedestrian	Camera captures text; on-device OCR reads it aloud
UC4 Navigate to a place	Pedestrian	Destination found, route computed, spoken turn-by-turn steps
UC5 Where am I?	Pedestrian	GPS and reverse geocoding speak the street and area

Use case	Actor	Main flow
UC6 Give a voice command	Pedestrian	Speech is matched to an action; online assistant if available
UC7 Send emergency SOS	Pedestrian	5-second countdown (cancellable), then SMS with location to contacts
UC8 Mark SOS as resolved	Helper / contact	SOS event status set to resolved
UC9 Manage emergency contacts	Pedestrian	Add, edit or remove contacts stored on the phone
UC10 Change settings	Pedestrian	Speech, vibration, detection source and AI options saved at once
UC11 Name objects with AI	Pedestrian (online)	Camera picture sent to Gemini; names shown on screen
UC12 Save pictures for training	Pedestrian, then developer	Unsure or taught pictures saved; labelled on the PC

Entity-Relationship Diagram

The entity-relationship diagram shows the data the system keeps. GuideAI has no user accounts, so the data belongs to the phone, identified by a random device identifier created at first launch. Settings and emergency contacts are stored on the phone. Only the SOS event is also stored on the server, linked to the phone by its device identifier. Navigation routes, text-reader history, and camera frames are not stored.

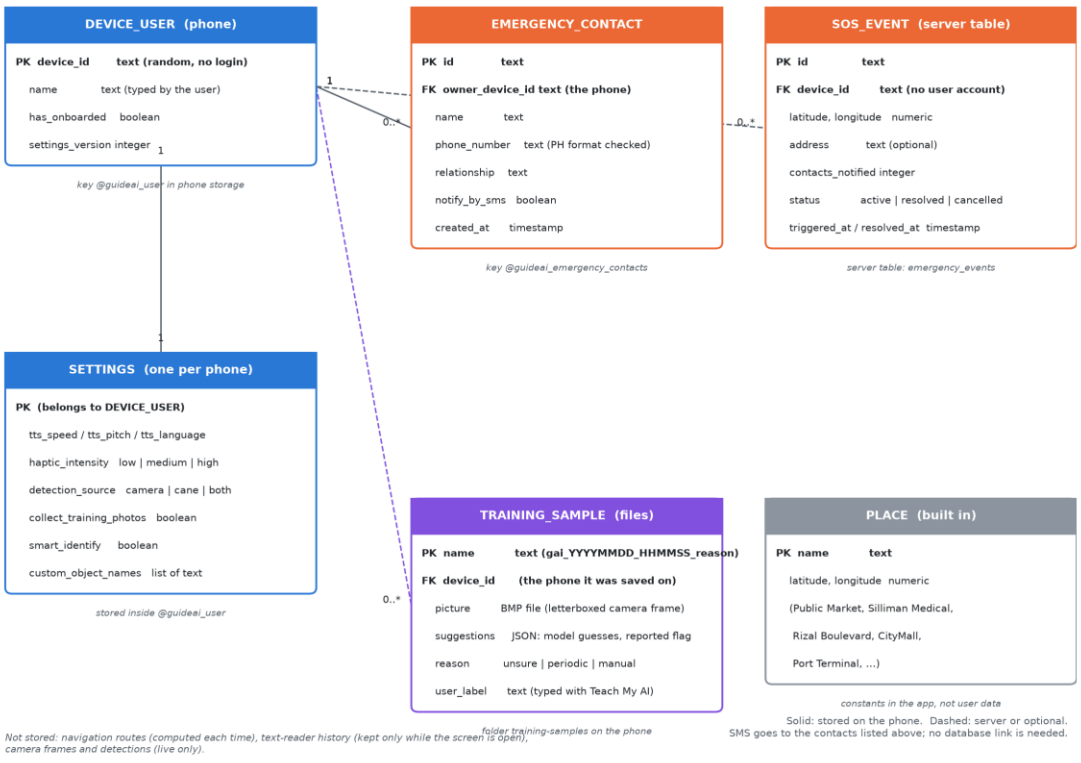


Figure 11. Entity-relationship diagram of GuideAI

APPENDIX H: CURRICULUM VITAE

[Proponent 1: photo, full name, address, contact number, e-mail, educational background, skills, and seminars attended]

[Proponent 2: photo, full name, address, contact number, e-mail, educational background, skills, and seminars attended]

[Proponent 3: photo, full name, address, contact number, e-mail, educational background, skills, and seminars attended]